MADD GROUP PROJECT

BudgetingApp - Android Development

Technical Implementation Report

Developer	
Module	BudgetActivity (Section 2.4)
Language	Java + XML (Android Studio)
Date	February 2026
Theme Colour	Blue (#1565C0)

1. Project Overview

The BudgetingApp is an Android mobile application designed to help users track income, categorize expenses, set budgets, and gain insights into their financial habits. The app is built to be intuitive, flexible, and capable of providing real-time feedback and recommendations to improve financial discipline and support goal achievement.

This document covers the implementation of Section 2.4 of the group project — the BudgetActivity module, assigned to Musembi Muia. It details the full development process from Android Studio setup to GitHub upload, including all files created, design decisions made, errors encountered, and how they were resolved.

1.1 Assignment Requirements

The following requirements were specified in the task allocation document:

- Files to work on: BudgetActivity.java, activity_budget.xml, GoalAdapter.java (confirmed as BudgetAdapter.java after team clarification)
- Purpose: Create and manage category budgets
- List of category budgets with progress bars (spent / limit)
- Create Budget card with category selector, amount input, and period/month selector
- Monthly overview at the top showing total budget vs total spent
- Tap a budget to edit or see transactions filtered by that category
- DB interactions: BudgetDao and TransactionDao for spent calculations
- Visual: Progress bars must use primary blue colour with clear percent labels

1.2 Technology Stack

Technology	Purpose
Java	Primary programming language for Android activities and logic
XML	UI layout design for all screens and dialogs
Room Database	Local SQLite database with DAO pattern
RecyclerView	Scrollable list of budget cards
Android Studio	Integrated Development Environment (IDE)
GitHub	Version control and team collaboration
Gradle (Kotlin DSL)	Build system and dependency management

2. Android Studio Setup

Since the GitHub repository was not cloned directly, a fresh Android Studio project was created and all code was written from scratch, designed to integrate seamlessly with the team's codebase during the Monday merge.

2.1 Creating a New Project

1. Opened Android Studio and selected New Project
2. Selected Empty Views Activity template
3. Configured project settings as follows:

Setting	Value
Project Name	BudgettingApp
Package Name	com.example.budgettingapp
Language	Java
Minimum SDK	API 24 (Android 7.0)
Build System	Gradle (Kotlin DSL)

4. Clicked Finish and waited for initial Gradle sync to complete

2.2 SDK Configuration

After project creation, Android Studio displayed a 'Project JDK is not defined' warning. This was resolved by:

5. Clicking the Setup SDK prompt that appeared at the top of the editor
6. Selecting Android API 36.0, extension level 17 Platform Android SDK from the dropdown
7. Waiting for Gradle to re-sync automatically

The Android SDK was chosen over JetBrains Runtime or Oracle JDK options because it is the correct SDK for Android development.

2.3 Adding Gradle Dependencies

The following dependencies were added to build.gradle.kts (Module: app) to support Room Database, RecyclerView, CardView, and Material Design components:

Dependency	Version	Purpose
androidx.room:room-runtime	2.6.1	Room database runtime
androidx.room:room-compiler	2.6.1	Room annotation processor
androidx.recyclerview:recyclerview	1.3.2	Scrollable budget list
androidx.cardview:cardview	1.0.0	Budget card UI components

com.google.android.material:material	1.11.0	Material Design buttons and styling
androidx.lifecycle:lifecycle-livedata-ktx	2.7.0	LiveData support

After adding dependencies, File > Sync Project with Gradle Files was clicked to download and apply them.

3. Package Structure

The project was organized into sub-packages following Android best practices and the structure defined in the group's task allocation document. All packages were created under the main package com.example.budgettingapp.

Package	Files	Purpose
Activities	BudgetActivity.java	Hosts all Activity classes — the main screen controllers
Adapters	BudgetAdapter.java	RecyclerView adapters that bind data to UI card layouts
DataBase	AppDatabase.java, BudgetDao.java, CategoryDao.java, TransactionDao.java	Room database singleton and all DAO interfaces
Models	Budget.java, Category.java	Room entity classes representing database tables

3.1 How Packages Were Created

8. In the Project panel, right-clicked on com.example.budgettingapp
9. Selected New > Package
10. Typed the package name (e.g. Activities) and pressed Enter
11. Repeated for Adapters, DataBase, and Models
12. Created Java class files inside each package using New > Java Class

4. Files Implemented

A total of 11 files were created to complete the BudgetActivity module. Each file is described below with its purpose and key implementation details.

4.1 Models

Budget.java

The Budget class is a Room @Entity that maps to the budgets table in the SQLite database. It stores the core budget data including the linked category, spending limit, and period.

Field	Type	Stored in DB	Description
id	int	Yes	Auto-generated primary key
categoryId	int	Yes	Links budget to a category
limitAmount	double	Yes	Maximum spending allowed
period	String	Yes	Monthly, Weekly, or Yearly
amountSpent	double	No (runtime)	Calculated from transactions at runtime
categoryName	String	No (runtime)	Fetched from Category table at runtime

Two convenience methods were also added: getUsagePercent() returns the percentage of budget used, and isOverBudget() returns true when spending exceeds the limit.

Category.java

The Category class is a Room @Entity mapping to the categories table. It stores category names (e.g. Food, Transport, Rent) that are used to classify budgets and transactions. This class will be shared with Eugine's IncomeExpenseActivity module.

4.2 Database Layer

BudgetDao.java

The BudgetDao interface is annotated with @Dao and provides all database operations for the budgets table. All methods run on background threads as required by Room.

Method	Operation	Description
insertBudget(Budget)	INSERT	Adds a new budget, replaces on conflict

updateBudget(Budget)	UPDATE	Updates an existing budget record
deleteBudget(Budget)	DELETE	Removes a budget from the database
getAllBudgets()	SELECT	Returns all budgets ordered by newest first
getAllBudgetsLive()	SELECT	LiveData version for reactive UI updates
getBudgetByCategory(int)	SELECT	Finds a budget for a specific category
getTotalMonthlyBudget()	SELECT	Sums all monthly budget limits (for MainActivity)

CategoryDao.java

Provides insert and fetch operations for the categories table. The getAllCategories() method is used by BudgetActivity to populate the category spinner in the Create Budget dialog.

TransactionDao.java

This DAO is a stub implementation that returns 0 for spent calculations. The real implementation will be provided by the team after the transactions table is created during the Monday merge. The stub ensures BudgetActivity compiles and runs without errors in the meantime.

AppDatabase.java

The Room database singleton that provides access to all DAOs. It uses a thread-safe getInstance() pattern to ensure only one database instance exists throughout the app lifecycle. This is a standalone version created for independent testing and will be replaced by the team's master AppDatabase during integration.

4.3 Adapter

BudgetAdapter.java

The BudgetAdapter extends RecyclerView.Adapter and is responsible for binding budget data to the item_budget.xml card layout. Each budget card displays the category name, a progress bar, the amount spent vs the limit, and a percentage label.

The progress bar colour changes dynamically based on spending level to provide clear visual feedback to the user:

Spending Level	Colour	Hex Code	Meaning
Below 70%	Blue	#1565C0	Spending is on track
70% to 89%	Yellow/Amber	#FFC107	Approaching budget limit

90% and above	Red	#F44336	At or over budget — warning

Clicking any budget card triggers the onBudgetClick callback which opens the Edit Budget dialog in BudgetActivity.

4.4 Activity

BudgetActivity.java

The main Activity class that ties all components together. It initializes the database, loads data on background threads using ExecutorService, and manages the UI on the main thread using runOnUiThread().

Key methods implemented:

- onCreate() — Initializes DB, binds views, sets month label, seeds test categories, then loads budgets
- loadBudgets() — Fetches all budgets from DB, attaches category names and spent amounts, updates the monthly overview card and RecyclerView
- showCreateBudgetDialog() — Inflates dialog_create_budget.xml, populates spinners, validates input, and saves new budget to DB
- showEditBudgetDialog() — Inflates same dialog with existing values pre-filled, allows update or delete with confirmation
- onBudgetTapped() — Callback from adapter that opens the edit dialog for the tapped budget

4.5 Layout Files

activity_budget.xml

The main screen layout built with CoordinatorLayout and NestedScrollView. Contains two primary sections: the monthly overview card at the top and the budget list below.

- AppBarLayout with blue Toolbar
- Blue CardView showing month label, Total Budget amount, Total Spent amount, and an overall progress bar
- Create Budget button in blue
- RecyclerView for the list of category budget cards

item_budget.xml

The layout for each individual budget card in the RecyclerView. Uses CardView with white background and elevation shadow. Contains the category name, a percentage label, a horizontal progress bar, and the KES spent / KES limit text.

dialog_create_budget.xml

A reusable dialog layout used for both creating and editing budgets. Contains a category Spinner, an amount EditText with decimal number input, and a period Spinner with Monthly, Weekly, and Yearly options.

5. Errors Encountered and Resolutions

Several errors were encountered during development. Each was identified, diagnosed, and resolved systematically. The table below documents all major errors and their fixes.

Error	File	Cause	Resolution
Project JDK is not defined	Project setup	No SDK selected after project creation	Selected Android API 36.0 SDK from the Setup SDK dropdown
Cannot resolve symbol 'item_budget'	BudgetAdapter.java	item_budget.xml layout file had not been created yet	Created item_budget.xml in res/layout with the correct view IDs
Cannot resolve symbol 'activity_budget'	BudgetActivity.java	activity_budget.xml did not exist	Created activity_budget.xml with all required view IDs
Cannot resolve symbol 'dialog_create_budget'	BudgetActivity.java	dialog_create_budget.xml was missing	Created dialog_create_budget.xml in res/layout
Unused parameters: categoryId, year, month	TransactionDao.java	Room does not allow parameters unused in the @Query SQL	Simplified the method to take no parameters and return 0 as a stub
Build failed — Gradle sync error	build.gradle.kts	Room and RecyclerView dependencies were missing	Added all required dependencies and clicked Sync Now
Category spinner empty in dialog	BudgetActivity.java	No categories existed in the database	Added a seed block in onCreate() to insert 7 default categories if the table is empty
Budget not saving after tap	BudgetActivity.java	Spinner was empty so getSelectedItemPosition() returned invalid index	Fixed by seeding categories first, then loading budgets after seed completes

6. Features Walkthrough

6.1 Monthly Overview Card

At the top of the screen is a blue card displaying the current month and year, the total budget limit across all categories, the total amount spent, and an overall progress bar. This gives the user an instant snapshot of their overall financial position for the month.

6.2 Create Budget

Tapping the Create Budget button opens a dialog with three input fields. The user selects a category from a spinner populated from the database, enters a budget amount in Kenyan Shillings, and selects a period (Monthly, Weekly, or Yearly). After tapping Save, the budget is inserted into the database and the list refreshes automatically with a success toast message.

6.3 Budget Cards with Progress Bars

Each budget is displayed as a card in a scrollable RecyclerView. The card shows the category name, a horizontal progress bar in blue showing spending progress, the exact KES spent vs KES limit amounts, and a percentage label. The progress bar and percentage label change colour from blue to yellow at 70% spending and to red at 90% spending, providing clear visual warnings to the user.

6.4 Edit and Delete Budget

Tapping any budget card opens an edit dialog with the current limit amount pre-filled and the current period pre-selected. The user can update the amount and period then tap Update to save changes. Tapping Delete triggers a confirmation dialog asking the user to confirm before the budget is permanently removed, preventing accidental deletions.

6.5 Background Threading

All database operations are performed on a background thread using Java's ExecutorService with a single-threaded executor. UI updates are then posted back to the main thread using runOnUiThread(). This ensures the app never blocks the main thread with database operations, which would cause the app to freeze or crash with an ANR (Application Not Responding) error.

7. Requirements Completion Status

Requirement	Status	Notes
BudgetActivity.java	Complete	Fully implemented with all features
activity_budget.xml	Complete	Blue themed layout with all required views
BudgetAdapter.java	Complete	GoalAdapter.java in spec confirmed as typo — BudgetAdapter is correct
List of category budgets with progress bars	Complete	RecyclerView with blue progress bars
Create Budget card (category, amount, period)	Complete	Dialog with spinners and validation
Monthly overview (total budget vs spent)	Complete	Blue card at top of screen
Tap to edit budget	Complete	Edit dialog with pre-filled values
BudgetDao interactions	Complete	Full CRUD operations implemented
TransactionDao for spent calculations	Stubbed	Returns 0 until team merges transactions table
Progress bars use primary blue	Complete	Blue #1565C0 with yellow/red warnings
Clear percent label on each card	Complete	Percentage shown top-right of each card

7.1 Notes for Team Integration

- TransactionDao.getSpentByCategory() is currently a stub returning 0. Replace with the real implementation that queries the transactions table after the team merge.
- The seed categories block in BudgetActivity.onCreate() should be removed after Eugine's CategoryDao integration is merged, as categories will then come from the IncomeExpenseActivity.
- AppDatabase.java is a standalone version. Replace with the team's master AppDatabase that includes all entity classes and DAOs from all modules.
- GoalAdapter.java listed under this section in the task document is a typo. GoalAdapter belongs to James Muritu's GoalsActivity (Section 2.5).

8. GitHub Upload Process

The completed code was uploaded to the team's GitHub repository using Android Studio's built-in Git integration. The following steps were followed:

8.1 Steps Followed

13. Enabled Git in Android Studio via VCS > Enable Version Control Integration
14. Staged all 59 project files by selecting Unversioned Files in the Commit panel
15. Wrote a descriptive commit message: 'Added BudgetActivity - Musembi's part'
16. Included integration notes in the commit message about stubs and seed data
17. Clicked Commit Anyway and Push to bypass style warnings
18. Confirmed the push in the Push Commits dialog
19. Verified the files appeared on the GitHub repository page
20. Added a README.md file directly through the GitHub website

8.2 Commit Message

```
Added BudgetActivity - Musembi's part
```

The repository showed '1 commit ahead' confirming the upload was successful.

9. Conclusion

The BudgetActivity module has been successfully implemented and uploaded to the team's GitHub repository. All core requirements from the task allocation document have been met, with the exception of the TransactionDao spent calculation which is intentionally stubbed pending the team merge.

The implementation follows Android best practices including background threading for all database operations, the DAO pattern for database access, proper separation of concerns across Activities, Adapters, Models, and Database layers, and a clean blue-themed UI consistent with the project's design specification.

The module is ready for integration with the team's codebase and the spent calculation functionality will activate automatically once the transactions table is available after the Monday merge session.

9.1 Key Achievements

- 11 files created covering all layers of the Android architecture
- Full CRUD operations for budgets implemented and tested on a physical OPPO device
- Colour-coded progress bars providing clear visual financial feedback
- All database operations running safely on background threads
- Code uploaded to GitHub with clear commit message and README documentation